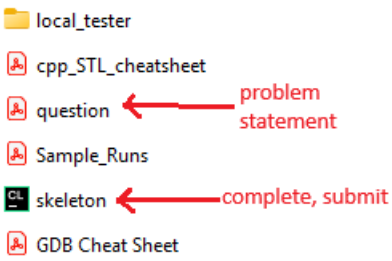
UPI Wallet System

Lab-test: EGC 211P Section A

Date: 23-09-2026, Time: 2:00pm – 3:30pm

READ THE INSTRUCTIONS CAREFULLY

Contents of InLab_test.zip



Edit the skeleton code `skeleton.cpp` to complete implementation and submit.

`skeleton.cpp` has commented method declarations per class. Uncomment one at a time, complete the implementation, test, then move to the next method.

By default, `skeleton.cpp` does not have any compilation error, but does not pass any testcase. So, if you submit it as it is ⇒ 0 marks.

Compilation error ⇒ 0 marks. So, if you are unable to fix compilation error for some code, comment it before submission.

DO NOT use bits/stdc++ header. If used, test scripts will reduce 50% of total earned points.

Domjudge testcases do not check your couts. But, log file contents are checked. So, copy-paste the logs from the question pdf; do not type since that is prone to introduce spelling errors.

Command to run local testcases: (in the folder where you have `skeleton.cpp`)

```
sh ./local_tester/local_test.sh
```

1 Application Overview

The application represents an in-memory UPI wallet system. The application allows users to register, log in, add money to a wallet, transfer money to another registered user, check their wallet balance, and log out. A user registers using a phone number and a name. The application automatically generates a UPI ID from the user's name. After logging in, a user can perform wallet operations. All important activities, including successful operations and failed operations, are written in a persistent log file.

2 Classes

The following classes are present in the design (some of them are already defined – except the constructor and destructor. You are free to add any members if you need.):

1. `UPIException`: Exception class to represent any kind of exception that can happen within the application. Already defined.
2. `Logger`: A class that can log any message passed to it to a particular log file. Already defined.
3. `Wallet`: Class that manages the balance of an account. Class declaration is given; Member names are self-explanatory (details given in Section 4).
4. `Account`: Class declaration is given; Member names are self-explanatory (details given in Section 5). You need to complete the method definitions.
5. `UPISystem`: Class declaration is given; you need to complete the method definitions (Section 6).

3 Command Enumerations [No Change Required]

Corresponding enums are defined to implement the following commands, which are used by `printMenu` function:

3.1 Main Menu Commands

`enum class MainMenuCmd`

Value	Command
1	REGISTER
2	LOGIN
3	EXIT

3.2 User Menu Commands

`enum class UserMenuCmd`

Value	Command
1	SEND_MONEY
2	ADD_MONEY
3	CHECK_BALANCE
4	LOGOUT

4 `Wallet` Class

Balance amount is kept as a private data member. Implement overloaded operators + and −, which perform credit and debit operations, respectively. A new wallet object should have an initial balance of 0.0. **Add constructors to this class as per your need**.

Throw Exceptions: The + and − operators throw exception when there is logical problems: trying to credit/debit non-positive amount, and trying to debit more amount than the balance. Below are the messages that are added as details of the exception (you may copy-paste the sentences and use as string argument to the exception, to get them as `e.what()` later in the caller class from the exception object `e`.) for logging.

Table 1: Registration UPIException Cases and corresponding Exception messages

UPIException Case	Message
Credit Amount is non-positive	`Invalid amount.`
Debit Amount is non-positive	`Invalid amount.`
Debit Amount exceeds balance	`Balance not sufficient!`

5 `Account` Class

Apart from the data members to capture name, phone number, UPI ID and a wallet, Account class has two *static constants*. Properly initialize them by following the details below:

1. `MAX_ADD_LIMIT` specifies the limit on adding balance to wallet *each time*. It's value is 20000.
2. `MAX_TXN_LIMIT` specifies the limit of transfer amount for each transaction. It's value is 10000.

5.1 Basic methods

Complete the definitions of the constructor and the getters. You may add other methods if needed.

5.2 `addMoney` Method

This method is for adding balance to the wallet. It accepts one Logger object as parameter, to log the internal exceptions and success of operations.

Throw Exceptions: Exception is thrown when the amount exceeds limit. Below are the messages that are added as details of the exception (you may copy-paste the sentences and use as string argument to the exception, to get them as `e.what()` from the exception object `e`.).

Table 2: Registration UPIException Cases and corresponding Exception messages

UPIException Case	Message
Amount exceeds `MAX_ADD_LIMIT`	`Add-money limit exceeded (max Rs.20000).`

Handle Success: For valid amount, wallet balance gets updated using wallet's + operator, and logging happens following this structure (replace with the correct attributes for the place-holders shown using `<angular brackets>`):

```
[<upiId>]: Added Rs.<amount> to <name>'s wallet. Balance: Rs.<balance>
```

Handle Exception : Exception (either raised from this method or from the methods which get called in this method) is caught to log the operation failures. Use the logger object to log the `what()` of the exceptions, in the following format:

```
[<upiId>]: Add-money failed: <e.what()>
```

5.3 `sendMoney` Method

This is to model the upi transfer action. Read the method declaration in the skeleton to understand the parameters.

Throw Exception: Exceptions are thrown when any of the following conditions happen, due to which, the operation cannot be completed:

Table 3: Transfer UPIException Cases and corresponding Exception messages

UPIException Case	**Message**
Amount to be transferred is greater than `MAX_TXN_LIMIT`	`Transaction limit exceeded (max Rs.10000).`
Receiver is the same as the sender	`Cannot send money to self.`

Handle Success : If validation succeeds,

1. First subtract the amount from the sender's balance, using the − operator of the wallet.
2. The success log follows this structure:

```
[<upiId>]: Rs.<amount> debited from <upiId>
```

3. then add the amount to the receiver's balance using the `addMoney` method of the receiver account;

Handle Exception: Exception is caught to log the operation failures. Use the logger object to log the `what()` of the exceptions, in the following format:

```
[<upiId>]: Send money failed: <e.what()>
```

5.4 `showBalance` Method

No logging is required for this method. Complete the definition to show balance in this format in the standard input, which also returns the wallet balance:

```
<name>'s balance: Rs.<balance>
```

6 `UPISystem` Class

This is the biggest class in the application, holding all of the core logic. It manages user registration to create accounts, account lookup, login, and logging. The class skeleton code already has A `Logger` initialized with the filename `upi_log.txt` – all your logs will be appended in this file.

6.1 Private Data Members

First, read the method descriptions given below to understand the data requirement. Then, define attributes to maintain Accounts, their phone numbers and UPI IDs. You may use any of STL classes, or even in-built arrays. Define getters, if necessary. There is no testcase for the getters because the internal attribute structure would vary from person to person. As long as your attributes can manage the method signatures given in the skeleton code, and implements the behaviors described below, it is fine.

6.2 `generateUpiId` Method (private, static)

This method already defined. It is a static method, which takes a name, converts into lower case and then creates a upiId for that name. E.g., upiId for a user with name `DeepikaPadukone` is `deepikapadukone@ok.iiitb`. Do not change this method.

6.3 `findAccount` Method (private)

Define private a method named findAccount, with a single parameter of string type. The parameter is a look-up key, which is either the phone number or UPI ID. This method is to look-up a registered account, so that money can be transferred to it. Once the account is found, the object/reference to it needs to be returned so that the caller can use it for transferring money.

Throw Exception: If no account can be found with the above identifier, throw exception with the following message:

```
[look-up key] Receiver not registered on UPI.
```

Handle Success: No logging is required for the success case, just return the identified account.

6.4 `registerUser` Method

This method takes name and phone number, gets an UPI ID generated internally, and creates one Account with these details. The new account remains saved in the internal data structures in the UPISystem.

Throw Exception: The following cases raises exception when trying to register a new user:

Table 4: Registration UPIException Cases and corresponding Exception messages

UPIException Case	Message
There is already one Account with the same phone number	`Phone number already registered.`
There is already one Account with the same UPI ID	`Generated UPI ID already in use. Try a different name.`

Handle Success: When Account creation succeeds, use the logger to log in the following structure:

```
[UPISystem]: Registered <name> | Phone: <phone> | UPI ID: <generated-UPI-ID>
```

Handle Exception: On failure case, following structure is for logging:

```
[UPISystem]: Registration failed: <e.what()>
```

6.5 `login` Method

This is for a user to login to the system. It accepts one identifier, which may be either the registered phone number or the exact name. The method should look-up for an existing Account matching the above details. If no account is found, throw an exception with the following message:

```
No such user <Identifier>. Check phone number or name.
```

The logger should have the following log:

```
[UPISystem]: Login failed: <e.what()>
```

On successful login, log the following:

```
[upiId]: Login successful.
```

6.6 `doUserAction` Method

This is the main method which handles all the user actions for a logged-in user. The method takes the logged-in user Account `(Account* currentUser)`, and the user choice `(int choice)`. The skeleton for each choice is given (corresponding enums in Section 3.2), along with their input actions. Fill-in the rest of the details (wiring with the internal data structures and Account objects) to perform the actions of sending money, adding money, checking balance and to logout.

7 Driver Program

7.1 `main` and `doApplicationJob` functions

These function implement the main application infinite loop. No change is required as these functions are completely defined. However, you need to read and understand the `doApplicationJob` function to understand the working of the code.

7.2 `doAppAction` function

This function gives the options to register a user, login and exit. The skeleton with the enum choices are given. You need to fill-in the wiring of the commands with object method calls. Note that this function has **bool** return type. It should return true when application should continue with the next choice. It should return false only when to exit the application, either due to user command, or due to some irrecoverable exception. Note that, UPIExceptions are not irrecoverable. They just indicate a particular user operation failed, but the application still should continue after logging the failure.